

ПРАКТИЧЕСКАЯ 9: КЛАССИФИКАЦИЯ ДАННЫХ

Ожидаемое время выполнения: 3 часа (1 час — логистическая регрессия, 1 час — KNN, 1 час — анализ результатов и оформление отчёта).

Цель практики

Освоить построение моделей классификации (логистическая регрессия и KNN) с использованием Python, R, а также интерпретацию результатов классификации через отчёты.

Задачи практики

1. Построить модель логистической регрессии:

- Python: использование библиотеки `sklearn` и `statsmodels`.
- R: функция `glm()` с семейством `binomial`.

2. Построить модель К-ближайших соседей (KNN):

- Python: библиотека `sklearn` (`KNeighborsClassifier`).
- R: использование пакета `class` или `caret`.

3. Выполнить прогноз на тестовых данных:

- Python и R: метрики оценки точности (`accuracy`, `precision`, `recall`).

4. Интерпретировать результаты моделей:

- Выявить важные переменные в логистической регрессии.
- Оценить точность моделей с помощью ROC-кривой, матрицы ошибок.
- Сравнить результаты между Python, R

Формат отчета

1. Введение:

- Цель и задачи работы.
- Краткое описание методов классификации: логистическая регрессия и KNN.

2. Результаты работы:

- Построение моделей:

- Python: код и визуализация результатов.
- R: скрипты с использованием `glm()` и KNN.
- Метрики оценки качества:
- Матрица ошибок, ROC-кривая, точность классификации.
- Таблица сравнения метрик между Python, R.
- Интерпретация:
- Какие переменные оказались важными?
- Какие выводы можно сделать на основе отчётов?

3. Заключение:

• Анализ удобства построения и интерпретации моделей классификации в Python, R.

• Рекомендации по выбору инструмента для решения задач классификации.

Дополнительные материалы к практической работе 9

Исходные данные: Набор данных для бинарной классификации (например, вероятность оттока клиентов, покупка продукта).

ПОШАГОВАЯ ИНСТРУКЦИЯ К ПРАКТИЧЕСКОЙ РАБОТЕ 9

ШАГ 1. ПОДГОТОВКА ДАННЫХ

1.1. Общие требования к данным

Структура: Должен быть набор признаков (колонки) и целевая переменная для бинарной классификации (значения 0/1, True/False и т. п.).

Очистка:

Удалите или обработайте пропущенные значения.

При наличии категориальных признаков (строк, категорий) используйте кодирование (One-Hot Encoding / dummy-переменные) либо убедитесь, что инструменты поддерживают работу с факторными переменными.

Разделение на обучающую и тестовую выборки:

Для более корректной оценки качества классификации необходимо иметь две части данных:

Train (для обучения),

Test (для проверки качества).

Обычно используют пропорцию 70/30, 80/20 или любую другую, оправданную вашей задачей.

1.2. Python (sklearn, statsmodels)

Импорт библиотек:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score,
recall_score, confusion_matrix, roc_curve, auc
import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
```

Загрузка и просмотр данных:

```
df = pd.read_csv('your_data.csv')
print(df.head())
print(df.info())
```

Разделение признаков и целевой переменной:

```
X = df.drop('Target', axis=1) # Предположим, что целевая колонка
называется 'Target'
y = df['Target']
```

One-Hot Encoding (если есть категориальные признаки):

```
X = pd.get_dummies(X, drop_first=True)
```

Разделение на train/test:

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

1.3. R (glm, классические библиотеки для KNN)

Загрузка библиотек:

```
# install.packages("caret") - пакет, содержащий множество
методов, включая KNN
# install.packages("e1071") - для метрик и доп. функций
# install.packages("ggplot2")
library(caret)
library(e1071)
library(ggplot2)
```

Импорт данных:

```
df <- read.csv("your_data.csv", header=TRUE,
stringsAsFactors=FALSE)
head(df)
```

Преобразование типов (если требуется):

```
str(df)
df$Target <- as.factor(df$Target) # если бинарная переменная в
виде 0/1
```

Разделение train/test (через caret):

```
set.seed(42)
```

```
trainIndex <- createDataPartition(df$Target, p=0.7, list=FALSE)
df_train <- df[trainIndex, ]
df_test <- df[-trainIndex, ]
```

ШАГ 2. ПОСТРОЕНИЕ МОДЕЛИ ЛОГИСТИЧЕСКОЙ РЕГРЕССИИ

2.1. Теоретические аспекты

Логистическая регрессия используется для бинарной классификации. Модель предсказывает вероятность принадлежности к классу «1»:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 \times X_1 + \dots + \beta_n \times X_n)}}$$

Интерпретация коэффициентов:

- Знак коэффициента (β_i) говорит о влиянии признака X_i на вероятность (положительное/отрицательное).
- Часто смотрят на коэффициенты в экспоненциальном виде (odds ratio).

2.2. Python (sklearn и statsmodels)

С использованием sklearn

```
logreg = LogisticRegression(max_iter=1000) # max_iter увеличен,
если много признаков
logreg.fit(X_train, y_train)

y_pred_logreg = logreg.predict(X_test)
```

Метрики качества

```
accuracy = accuracy_score(y_test, y_pred_logreg)
precision = precision_score(y_test, y_pred_logreg)
recall = recall_score(y_test, y_pred_logreg)
print(f"Accuracy: {accuracy:.2f}, Precision: {precision:.2f},
Recall: {recall:.2f}")
```

С использованием statsmodels (для получения coef, p-value)

```
X_train_sm = sm.add_constant(X_train) # добавляем константу
model_sm = sm.Logit(y_train, X_train_sm).fit()
print(model_sm.summary())

# Предсказания
X_test_sm = sm.add_constant(X_test)
y_pred_prob = model_sm.predict(X_test_sm)
y_pred_class = (y_pred_prob >= 0.5).astype(int)
```

2.3. R (glm с семейством binomial)

```
# Предположим, что целевая переменная - Target, а остальные -
# предикторы
model_log <- glm(Target ~ ., data=df_train, family="binomial")
summary(model_log)
```

Предсказание и метрики

```
pred_prob <- predict(model_log, df_test, type="response")
pred_class <- ifelse(pred_prob >= 0.5, 1, 0)
conf_matrix <- table(pred_class, df_test$Target)
conf_matrix

accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
accuracy
```

ШАГ 3. ПОСТРОЕНИЕ МОДЕЛИ KNN (К-БЛИЖАЙШИХ СОСЕДЕЙ)

3.1. Теоретические аспекты KNN

Идея: При классификации нового объекта смотрят на его K ближайших соседей в пространстве признаков. Класс, наиболее часто встречающийся среди соседей, присваивается объекту.

Особенности:

- Нужно выбрать K.

- Желательно масштабировать данные (чтобы признаки с большими значениями не доминировали в расстояниях).

3.2. Python (sklearn)

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)
```

Метрики

```
accuracy_knn = accuracy_score(y_test, y_pred_knn)
precision_knn = precision_score(y_test, y_pred_knn)
recall_knn = recall_score(y_test, y_pred_knn)
print(f"KNN -> Accuracy: {accuracy_knn:.2f}, Precision: {precision_knn:.2f}, Recall: {recall_knn:.2f}")
```

3.3. R (caret или class)

Использование caret

```
control <- trainControl(method="cv", number=5) # кросс-валидация
tuneGrid <- expand.grid(k=seq(3, 15, 2)) # поиск по k

model_knn <- train(Target ~ ., data=df_train, method="knn",
                  trControl=control, tuneGrid=tuneGrid)
model_knn
```

Предсказание

```
pred_knn <- predict(model_knn, newdata=df_test)
confusionMatrix(pred_knn, df_test$Target)
```

ШАГ 4. ОЦЕНКА КАЧЕСТВА КЛАССИФИКАЦИИ

4.1. Матрица ошибок (Confusion Matrix)

Состоит из 4 ячеек: True Positive (TP), False Positive (FP), False Negative (FN), True Negative (TN).

Позволяет вычислить основные метрики точности:

- $\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$
- $\text{Precision} = TP / (TP + FP)$
- $\text{Recall} = TP / (TP + FN)$
- $\text{F1-score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$

4.2. ROC-кривая и AUC

ROC-кривая: график чувствительности (TPR) против (1 – специфичность) (FPR) при варьировании порога классификации.

AUC: площадь под ROC-кривой, чем ближе к 1 — тем лучше модель.

Python

```
y_pred_prob_logreg = logreg.predict_proba(X_test)[:,-1] # вероятности для класса 1
fpr, tpr, _ = roc_curve(y_test, y_pred_prob_logreg)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f"LogReg (AUC={roc_auc:.2f})")
plt.plot([0,1],[0,1], 'r--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC-кривая")
plt.legend()
plt.show()
```


ШАГ 5. ИНТЕРПРЕТАЦИЯ И СРАВНЕНИЕ РЕЗУЛЬТАТОВ

Интерпретация логистической регрессии:

- Посмотрите на знаки и величины коэффициентов (β_i) и соответствующие p-значения (особенно, если модель строится в statsmodels).
- Опишите, какие переменные оказывают наибольшее влияние на вероятность принадлежности к классу 1.

KNN:

- Оцените, каково оптимальное K (если вы делали перебор).
- Обратите внимание на чувствительность к масштабу данных.
- Сравнительные метрики (Accuracy, Precision, Recall, AUC):
- Составьте таблицу, где для каждой модели (Логистическая регрессия, KNN) и для каждого инструмента (Python, R) укажите полученные значения метрик.

Различия в инструментах:

Python (гибкость, расширенные библиотеки, контроль гиперпараметров).

R (статистические методы, удобство моделирования через glm, caret).

КЛЮЧЕВЫЕ МОМЕНТЫ ДЛЯ УСПЕШНОГО ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

Разделение на train/test: обязательно покажите, что вы оценивали качество модели на данных, которые не использовались для обучения.

Масштабирование данных (особенно для KNN): не забудьте упомянуть важность одинаковых масштабов признаков.

Метрики: опишите, почему вы используете конкретные метрики (Accuracy, Precision, Recall, F1, ROC-AUC).

Интерпретация логистической регрессии: обратите внимание на коэффициенты, знаки, статистическую значимость.

Сравнение: сформируйте четкую таблицу итоговых результатов и сравните их для Python, R.

Таким образом, следуя пошаговым указаниям, вы сможете построить две разные модели классификации (логистическая регрессия и KNN), сравнить их качество в двух средах (Python, R), а также сделать выводы о наиболее эффективном методе и инструменте для конкретной задачи.